



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

IT4025E - CRYPTOGRAPHY

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

Introduction to Number Theory

Authors:

Student 1 : Ha Phuong Linh - 20235598

Student 2 : Hoang Danh Nhat - 20230095

Date: June 20, 2026

Abstract

This report provides a comprehensive overview of the fundamental number theory concepts essential for modern cryptography [1]. It covers the core principles of modular arithmetic, group structures, and key theorems such as Fermat's and Euler's. Furthermore, it analyzes the computation of quadratic residues and modular roots, evaluates the time complexity of large integer arithmetic algorithms, and classifies the tractability of various computational problems. By delineating the boundary between easy computations and intractable problems—namely the Discrete Logarithm and Integer Factoring—this document establishes the mathematical foundation underlying public-key encryption, digital signatures, and secure key exchange protocols. This report provides a comprehensive overview of the fundamental number theory concepts essential for modern cryptography. It covers the core principles of modular arithmetic, group structures, and key theorems such as Fermat's and Euler's. Furthermore, it analyzes the computation of quadratic residues and modular roots, evaluates the time complexity of large integer arithmetic algorithms, and classifies the tractability of various computational problems. By delineating the boundary between easy computations and intractable problems—namely the Discrete Logarithm and Integer Factoring—this document establishes the mathematical foundation underlying public-key encryption, digital signatures, and secure key exchange protocols.

1 Introduction

We will use a bit of number theory to construct secure cryptographic systems. Specifically, these theoretical concepts are utilized to construct:

- Key exchange protocols.
- Digital signatures.
- Public-key encryption.

Throughout this report, several standard mathematical notations will be frequently used:

- N denotes a positive integer. N often denotes an n -bit positive integer.
- p (and q) denotes a prime. For practical applications, we often want to generate a large random prime, such as a prime p of length 1024 bits ($p \approx 2^{1024}$).
- $Z_N = \{0, 1, 2, \dots, N - 1\}$ denotes the set of integers modulo N .

2 Arithmetic Modulo Primes

2.1 Basic Concepts and Linear Equations

Let p be a prime number. In cryptographic applications, p is typically a large prime, often on the order of 1024 bits ($p \approx 2^{1024}$). We define the set of integers modulo p as:

$$\mathbb{Z}_p = \{0, 1, 2, \dots, p-1\} \quad (1)$$

The elements of $\mathbb{Z}_p$ form a finite field under addition and multiplication modulo p . For any $x, y \in \mathbb{Z}_p$, their sum and product are uniquely defined within $\mathbb{Z}_p$ via modular reduction.

2.1.1 Modular Inversion

An element $x \in \mathbb{Z}_p$ is said to be invertible if there exists an element $a \in \mathbb{Z}_p$ such that:

$$a \cdot x \equiv 1 \pmod{p} \quad (2)$$

The element a is the unique modular inverse of x and is denoted by x^{-1} . A fundamental property of $\mathbb{Z}_p$ is that every non-zero element is invertible. Thus, the set of invertible elements, denoted as $\mathbb{Z}_p^*$, is given by:

$$\mathbb{Z}_p^* = \mathbb{Z}_p \setminus \{0\} = \{1, 2, \dots, p-1\} \quad (3)$$

The size of this set is $|\mathbb{Z}_p^*| = p-1$.

Special Inverses and General Inversion Algorithm

Example 1: In $\mathbb{Z}_5$, the inverse of 3 is 2 because $2 \cdot 3 = 6 \equiv 1 \pmod{5}$. Hence, $3^{-1} \equiv 2 \pmod{5}$.

Theorem 1 (Inverse of 2): For any odd prime p , the modular inverse of 2 in $\mathbb{Z}_p$ is given by $\frac{p+1}{2}$.

Proof 1

We need to show that $2 \cdot \left(\frac{p+1}{2}\right) \equiv 1 \pmod{p}$. Direct algebraic multiplication yields:

$$2 \cdot \left(\frac{p+1}{2}\right) = p+1 \quad (4)$$

Taking the result modulo p :

$$p+1 \equiv 0+1 \equiv 1 \pmod{p} \quad (5)$$

Since p is an odd prime, $p+1$ is guaranteed to be an even integer, which implies that $\frac{p+1}{2}$ is a valid integer. Furthermore, because $p > 2$, it holds that $1 \leq \frac{p+1}{2} \leq p-1$, meaning it safely belongs to $\mathbb{Z}_p^*$. This completes the proof.

2.1.2 Fermat's Little Theorem and Inversion

Theorem 2 (Fermat's Little Theorem): For any prime p and any element $g \in \mathbb{Z}_p^*$,

$$g^{p-1} \equiv 1 \pmod{p} \quad (6)$$

Application to Inversion: Fermat's Little Theorem provides an explicit formula for computing the inverse of any non-zero element $x \in \mathbb{Z}_p$. By multiplying both sides of $x \cdot x^{-1} \equiv 1 \pmod{p}$ by x^{p-2} , we can rewrite the expression as:

$$x \cdot x^{p-2} \equiv x^{p-1} \equiv 1 \pmod{p} \quad (7)$$

It follows directly that:

$$x^{-1} \equiv x^{p-2} \pmod{p} \quad (8)$$

Although this algorithm is conceptually simple to implement using modular exponentiation, it requires up to $2 \log_2 p$ modular multiplications. Therefore, it is asymptotically less efficient than the Extended Euclidean Algorithm, which computes the inverse in quadratic time $O(n^2)$ relative to the bit length n .

2.1.3 Solving Linear Equations

Using the existence of modular inverses, any linear equation of the form $a \cdot x \equiv b \pmod{p}$ (where $a \neq 0$) can be solved uniquely within $\mathbb{Z}_p$:

$$x \equiv b \cdot a^{-1} \equiv b \cdot a^{p-2} \pmod{p} \quad (9)$$

2.2 The Algebraic Structure of $\mathbb{Z}_p^*$

The set $\mathbb{Z}_p^*$ forms a multiplicative group of order $p - 1$. The efficiency and security of many public-key cryptosystems rely heavily on its underlying algebraic properties.

2.2.1 Cyclic Groups and Generators

Theorem 3: The multiplicative group $\mathbb{Z}_p^*$ is a cyclic group.

Proof 2 (Contextual Note)

The property that $\mathbb{Z}_p^$ is cyclic stems from the fact that $\mathbb{Z}_p$ is a finite field, and any finite subgroup of the multiplicative group of a field is cyclic.*

This cyclicity implies that there exists an element $g \in \mathbb{Z}_p^*$ whose successive powers generate the entire group:

$$\mathbb{Z}_p^* = \{1, g, g^2, g^3, \dots, g^{p-2}\} \quad (10)$$

Such an element g is called a *generator* or a *primitive root* of $\mathbb{Z}_p^*$. We let G^* denote the set of all generators of a finite cyclic group G .

Example 2: Consider $\mathbb{Z}_7^* = \{1, 2, 3, 4, 5, 6\}$. Let us evaluate the powers of 3:

$$\begin{array}{ll} 3^1 \equiv 3 \pmod{7} & 3^4 \equiv 4 \pmod{7} \\ 3^2 \equiv 2 \pmod{7} & 3^5 \equiv 5 \pmod{7} \\ 3^3 \equiv 6 \pmod{7} & 3^6 \equiv 1 \pmod{7} \end{array}$$

Thus, $\{1, 3, 3^2, 3^3, 3^4, 3^5\} \equiv \{1, 3, 2, 6, 4, 5\} = \mathbb{Z}_7^*$. This demonstrates that 3 is a valid generator of $\mathbb{Z}_7^*$.

Conversely, not every element acts as a generator. For instance, evaluating the powers of 2 in $\mathbb{Z}_7^*$:

$$\begin{array}{l} 2^1 \equiv 2 \pmod{7} \\ 2^2 \equiv 4 \pmod{7} \\ 2^3 \equiv 1 \pmod{7} \end{array}$$

The subset generated by 2 is restricted to $\{1, 2, 4\} \neq \mathbb{Z}_7^*$, meaning 2 is not a generator.

2.2.2 Order of an Element and Lagrange's Theorem

Definition (Order): The order of an element $g \in \mathbb{Z}_p^*$, denoted as $\text{order}_p(g)$, is the smallest positive integer a such that:

$$g^a \equiv 1 \pmod{p} \quad (11)$$

From the previous examples, it is clear that $\text{order}_7(3) = 6$ and $\text{order}_7(2) = 3$.

Theorem 4 (Lagrange's Theorem for $\mathbb{Z}_p^*$ Unique): For any $g \in \mathbb{Z}_p^*$, the order of the element must divide the total order of the group, which is $p - 1$:

$$\text{order}_p(g) \mid (p - 1) \quad (12)$$

Proof of Fermat's Little Theorem as a Corollary: Using Lagrange's theorem, Fermat's Little Theorem can be cleanly derived. Let $k = \text{order}_p(g)$. By Theorem 4, k divides $p - 1$, meaning there exists a positive integer m such that:

$$p - 1 = k \cdot m \quad (13)$$

Now, evaluating the expression g^{p-1} modulo p :

$$g^{p-1} = g^{k \cdot m} = (g^k)^m \quad (14)$$

By definition, since k is the order of g , we have $g^k \equiv 1 \pmod{p}$. Substituting this target value back into the equation yields:

$$(g^k)^m \equiv (1)^m \equiv 1 \pmod{p} \quad (15)$$

This concludes the formal deduction of Fermat's Little Theorem from group-theoretic principles.

2.2.3 Efficient Computation of Element Order

When the prime factorization of $p - 1$ is fully known, expressed as $p - 1 = q_1^{e_1} q_2^{e_2} \dots q_r^{e_r}$, there exists an efficient deterministic algorithm to calculate $\text{order}_p(g)$ for any element. Instead of exhaustively searching all divisors, the algorithm iteratively reduces the exponent $p-1$ by testing factors q_i . If $g^{(p-1)/q_i} \equiv 1 \pmod{p}$, the exponent can be safely divided by q_i , significantly minimizing the computation cost to a sequence of efficient modular exponentiations.

2.3 Quadratic Residues and Equations

2.3.1 Square Roots in $\mathbb{Z}_p$

Definition (Square Root): The square root of an element $x \in \mathbb{Z}_p$ is an element $y \in \mathbb{Z}_p$ such that $y^2 \equiv x \pmod{p}$. This relationship is conventionally denoted by the notation $\sqrt{x} \equiv y \pmod{p}$.

Example: In $\mathbb{Z}_7$, we write $\sqrt{2} \equiv 3 \pmod{7}$ because $3^2 = 9 \equiv 2 \pmod{7}$. However, $\sqrt{3} \pmod{7}$ does not exist in $\mathbb{Z}_7$.

How many square roots does $x \in \mathbb{Z}_p$ have?

A critical property of $\mathbb{Z}_p$ is that an element can have either zero or exactly two square roots. Suppose an element $x \in \mathbb{Z}_p$ has two square roots a and b , i.e.,

$$x \equiv a^2 \equiv b^2 \pmod{p}.$$

Then,

$$a^2 - b^2 \equiv (a - b)(a + b) \equiv 0 \pmod{p}.$$

Because $\mathbb{Z}_p$ is an integral domain, this implies

$$a - b \equiv 0 \pmod{p}$$

or

$$a + b \equiv 0 \pmod{p},$$

so $b = a$ or $b = -a$. Therefore, if a is a square root of x , then $-a$ is also a square root, and these are the only possible square roots.

2.3.2 Quadratic Residues (QR)

Definition (Quadratic Residue): An element $x \in \mathbb{Z}_p^*$ is called a Quadratic Residue (QR) if it has a valid square root in $\mathbb{Z}_p$.

Theorem 5 (Euler's Criterion for Quadratic Residues): An element $x \in \mathbb{Z}_p^*$ is a quadratic residue if and only if

$$x^{(p-1)/2} \equiv 1 \pmod{p}.$$

To understand why, recall that $\mathbb{Z}_p^*$ is a cyclic group, so every element can be written as $x = g^k$ for some generator g .

Consider the element

$$a = g^{(p-1)/2}.$$

Squaring both sides gives

$$a^2 = \left(g^{(p-1)/2}\right)^2 = g^{p-1} \equiv 1 \pmod{p}$$

by Fermat's Little Theorem. Thus, a is a square root of 1.

Now solve

$$y^2 \equiv 1 \pmod{p}.$$

We have

$$y^2 - 1 = (y - 1)(y + 1) \equiv 0 \pmod{p}.$$

Since $\mathbb{Z}_p$ is an integral domain, this implies

$$y = 1 \quad \text{or} \quad y = -1.$$

Therefore,

$$g^{(p-1)/2} \in \{1, -1\}.$$

Because g is a generator, its order is $p - 1$, so we cannot have

$$g^{(p-1)/2} = 1,$$

otherwise the order of g would divide $(p - 1)/2$. Hence,

$$g^{(p-1)/2} = -1.$$

Now let $x = g^k$. Then

$$x^{(p-1)/2} = (g^k)^{(p-1)/2} = (g^{(p-1)/2})^k = (-1)^k.$$

If k is even, then

$$x = g^{2m} = (g^m)^2,$$

so x is a quadratic residue and

$$x^{(p-1)/2} = 1.$$

If k is odd, then x is not a quadratic residue and

$$x^{(p-1)/2} = -1.$$

Example: Quadratic residues modulo 7

The nonzero elements of $\mathbb{Z}_7$ are

$$1, 2, 3, 4, 5, 6.$$

Compute their squares:

$$1^2 \equiv 1, \quad 2^2 \equiv 4, \quad 3^2 \equiv 2, \quad 4^2 \equiv 2, \quad 5^2 \equiv 4, \quad 6^2 \equiv 1 \pmod{7}.$$

Therefore, the quadratic residues modulo 7 are

$$\{1, 2, 4\}.$$

For example:

$$3^2 = 9 \equiv 2 \pmod{7}$$

and

$$4^2 = 16 \equiv 2 \pmod{7},$$

so 2 has square roots 3 and 4 modulo 7.

However, no element squares to 3 modulo 7, so 3 is not a quadratic residue.

Euler's criterion confirms this:

$$2^{(7-1)/2} = 2^3 = 8 \equiv 1 \pmod{7}$$

so 2 is a quadratic residue, while

$$3^{(7-1)/2} = 3^3 = 27 \equiv 6 \equiv -1 \pmod{7}$$

so 3 is not a quadratic residue.

Legendre Symbol: To determine whether an element is a quadratic residue without explicitly searching for square roots, we define the **Legendre symbol** for $x \in \mathbb{Z}_p$ as

$$\left(\frac{x}{p}\right) = \begin{cases} 1 & \text{if } x \text{ is a quadratic residue modulo } p, \\ -1 & \text{if } x \text{ is a quadratic non-residue modulo } p, \\ 0 & \text{if } x \equiv 0 \pmod{p}. \end{cases}$$

Euler's criterion provides an efficient way to compute this value:

$$\left(\frac{x}{p}\right) \equiv x^{(p-1)/2} \pmod{p}.$$

Since $x^{(p-1)/2}$ can only evaluate to 1 or -1 for nonzero elements, this immediately tells us whether x has a square root modulo p .

The connection becomes especially clear when expressing elements using a generator g of $\mathbb{Z}_p^*$. Any nonzero element can be written as

$$x = g^r.$$

Recall that quadratic residues are exactly the even powers of g . Therefore, $x = g^r$ is a quadratic residue if and only if r is even. Consequently, the Legendre symbol reveals the parity of the exponent:

$$\left(\frac{x}{p}\right) = \left(\frac{g^r}{p}\right) = (-1)^r.$$

Thus,

$$\left(\frac{g^r}{p}\right) = \begin{cases} 1 & r \text{ even (quadratic residue)} \\ -1 & r \text{ odd (quadratic non-residue)} \end{cases}$$

““latex

2.3.3 Computing Square Roots

When the prime satisfies

$$p \equiv 3 \pmod{4},$$

we can compute square roots modulo p using a simple deterministic formula.

Suppose $x \in \mathbb{Z}_p$ is a **quadratic residue** (QR), meaning that x already has a square root modulo p . Then a square root of x is given by

$$a \equiv x^{(p+1)/4} \pmod{p}.$$

Proof 3

Since x is assumed to be a quadratic residue, Euler's criterion gives

$$x^{(p-1)/2} \equiv 1 \pmod{p}.$$

Let

$$a = x^{(p+1)/4}.$$

We verify that a is indeed a square root of x :

$$a^2 = \left(x^{(p+1)/4}\right)^2 = x^{(p+1)/2} = x \cdot x^{(p-1)/2} = x \pmod{p}.$$

Thus, a is a valid square root of x .

Example (modulo 7).

Since

$$7 \equiv 3 \pmod{4},$$

the formula applies.

Consider $x = 2$. Because 2 is a quadratic residue modulo 7, we compute

$$a = 2^{(7+1)/4} = 2^2 = 4 \pmod{7}.$$

Checking,

$$4^2 = 16 \equiv 2 \pmod{7},$$

so the square roots of 2 modulo 7 are

$$4 \quad \text{and} \quad -4 \equiv 3 \pmod{7}.$$

However, the formula only works for quadratic residues. For example, 3 is not a QR modulo 7. Applying the formula gives

$$3^{(7+1)/4} = 3^2 = 2 \pmod{7},$$

but

$$2^2 = 4 \not\equiv 3 \pmod{7}.$$

This fails because 3 has no square root modulo 7. ““

2.3.4 Solving Quadratic Equations

With the ability to compute square roots, we can solve general quadratic equations in $\mathbb{Z}_p$ of the form $ax^2 + bx + c \equiv 0 \pmod{p}$.

The solution exists if and only if the discriminant $\Delta = b^2 - 4ac$ is a QR in $\mathbb{Z}_p$. If this condition holds, we find the square root $\sqrt{\Delta} \pmod{p}$ using the algorithms described above, and recover the roots using the standard quadratic formula:

$$x_{1,2} \equiv (-b \pm \sqrt{\Delta}) \cdot (2a)^{-1} \pmod{p} \quad (16)$$

2.4 Computing in $\mathbb{Z}_p$

When deploying cryptographic systems in practice, the prime p is typically extremely large (e.g., 1024 to 3072 bits long). Since standard hardware architectures generally feature 32-bit or 64-bit registers, these large integers cannot be processed natively in a single CPU instruction.

2.4.1 Bignum Representation and Basic Arithmetic

To process these large numbers, elements of $\mathbb{Z}_p$ are represented as arrays of machine words (often referred to as buckets or blocks), where each bucket is 32 or 64 bits long depending on the processor's chip size.

- **Addition and Subtraction:** Adding or subtracting two elements $x, y \in \mathbb{Z}_p$ is performed word-by-word with carry propagation. This operation runs in linear time $O(n)$ with respect to the bit length n of p .
- **Multiplication:** Naive multiplication takes quadratic time $O(n^2)$. However, advanced algorithms like Karatsuba can reduce the asymptotic bound to $O(n^{\log_2 3}) \approx O(n^{1.585})$. In most implementations, multiplication operates in roughly $O(n^{1.7})$ time.
- **Inversion:** Using the Extended Euclidean Algorithm computes the inverse $x^{-1} \pmod{p}$ efficiently in quadratic time $O(n^2)$.

2.4.2 Modular Exponentiation: Repeated Squaring

Computing $x^r \pmod{p}$ for a massive exponent r (e.g., a 1024-bit integer) is computationally impossible if executed naively via $r - 1$ sequential multiplications. Instead, cryptographic systems utilize the *Repeated Squaring* algorithm, which exploits the binary representation of the exponent:

$$r = (r_k r_{k-1} \dots r_1 r_0)_2 = \sum_{i=0}^k r_i \cdot 2^i \quad (17)$$

where $r_i \in \{0, 1\}$ and $k = \lfloor \log_2 r \rfloor$. The algorithm iteratively squares the base and multiplies the accumulated result by x whenever $r_i = 1$:

$$x^r \equiv \prod_{i=0}^k \left(x^{2^i}\right)^{r_i} \pmod{p} \quad (18)$$

This method guarantees that the exponentiation $x^r \pmod{p}$ can be evaluated using at most $2 \log_2 r$ modular multiplications.

Pre-computation Optimization: When the base x is fixed (a common scenario in digital signature generation), we can pre-compute a table of powers of x . By utilizing a window size of w bits (e.g., $w = 5$), the table stores $(2^w/w) \cdot \log_2 r$ group elements. This space-time tradeoff reduces the online computation time to approximately $(\log_2 r)/w$ multiplications.

2.5 Summary: Tractability of Problems in $\mathbb{Z}_p$

The fundamental security of public-key cryptographic constructs relies entirely on the one-way nature of certain mathematical operations. We classify problems in $\mathbb{Z}_p$ into two categories: tractable (easy) and intractable (hard). Let p be a sufficiently large prime (e.g., 1024 bits).

2.5.1 Tractable (Easy) Problems

The following operations can be executed efficiently in polynomial time:

- Generating a random element, alongside basic addition and multiplication.
- Computing modular exponentiation $g^r \pmod{p}$ for arbitrarily large r .
- Computing the inverse of an element and solving linear systems.
- Testing if an element is a Quadratic Residue (QR) and computing its square root (if it is a QR).
- Solving general polynomial equations of degree d in time polynomial relative to d .

2.5.2 Intractable (Hard) Problems

The following problems are widely believed to be computationally infeasible using classical computers. They form the mathematical bedrock of discrete logarithm-based cryptography:

- **Discrete Logarithm Problem (DLOG):** Let g be a generator of $\mathbb{Z}_p^*$. Given a target value $x \in \mathbb{Z}_p^*$, find the integer r such that $x \equiv g^r \pmod{p}$.
- **Diffie-Hellman Problem (DH):** Let g be a generator. Given two public values $x \equiv g^{r_1} \pmod{p}$ and $y \equiv g^{r_2} \pmod{p}$, compute the shared secret $z \equiv g^{r_1 r_2} \pmod{p}$ without prior knowledge of r_1 or r_2 .

3 Arithmetic Modulo Composites

In practical cryptographic applications, specifically in the RSA cryptosystem, we frequently operate over a composite modulus n . We generally assume that n is a large integer (e.g., 1024 to 3072 bits long) constructed as the product of two distinct, large primes of roughly equal size: $n = p \cdot q$.

We define the set of integers modulo n as:

$$\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\} \quad (19)$$

3.1 Basic Arithmetic and Inversion

Addition and multiplication in $\mathbb{Z}_n$ are performed similarly to $\mathbb{Z}_p$. However, unlike in prime fields, not every non-zero element in $\mathbb{Z}_n$ has a modular inverse.

Condition for Invertibility: The inverse of an element $x \in \mathbb{Z}_n$ is an element $y \in \mathbb{Z}_n$ such that $x \cdot y \equiv 1 \pmod{n}$. This inverse exists if and only if x and n are relatively prime, meaning:

$$\gcd(x, n) = 1 \quad (20)$$

Extended Euclidean Algorithm: To efficiently find the inverse, we utilize the Extended Euclidean Algorithm. If $\gcd(x, n) = 1$, the algorithm efficiently constructs two integers $a, b \in \mathbb{Z}$ such that $a \cdot x + b \cdot n = 1$. Reducing this equation modulo n eliminates the $b \cdot n$ term, leaving:

$$a \cdot x \equiv 1 \pmod{n} \quad (21)$$

Thus, $a \equiv x^{-1} \pmod{n}$. This algorithm operates in $O(n^2)$ time and is significantly more efficient than Fermat-based inversion algorithms. With the ability to invert elements, any linear equation $a \cdot x \equiv b \pmod{n}$ can be solved as $x \equiv b \cdot a^{-1} \pmod{n}$.

3.2 Euler's Totient Function and Theorem

We let $\mathbb{Z}_n^*$ denote the set of all invertible elements in $\mathbb{Z}_n$. The size of this set is of paramount importance and is given by Euler's Totient Function, denoted as $\phi(n)$.

For a prime p , $\phi(p) = p - 1$. For a composite modulus $n = p \cdot q$, the function is multiplicative, yielding:

$$\phi(n) = \phi(p) \cdot \phi(q) = (p - 1)(q - 1) = n - p - q + 1 \quad (22)$$

Theorem 6 (Euler's Theorem): Generalizing Fermat's Little Theorem to composite moduli, Euler's theorem states that for all $a \in \mathbb{Z}_n^*$:

$$a^{\phi(n)} \equiv 1 \pmod{n} \quad (23)$$

This theorem forms the mathematical foundation of the RSA encryption and decryption mechanism.

3.3 The Chinese Remainder Theorem (CRT)

The Chinese Remainder Theorem (CRT) is a powerful structural tool that establishes an isomorphism between the ring $\mathbb{Z}_n$ and the direct product of the rings of its prime factors $\mathbb{Z}_p \times \mathbb{Z}_q$. When the prime factorization of $n = p \cdot q$ is known, CRT allows us to break down complex computations (such as calculating square roots or solving high-degree polynomial equations) in $\mathbb{Z}_n$. The standard procedure involves:

1. Solving the equation modulo p to find roots in $\mathbb{Z}_p$.
2. Solving the equation modulo q to find roots in $\mathbb{Z}_q$.
3. Using CRT to seamlessly combine these solutions into the final roots modulo n .

Example (Solving a quadratic equation using CRT).

Consider the equation

$$x^2 \equiv 4 \pmod{35},$$

where

$$35 = 5 \cdot 7.$$

Rather than solving directly modulo 35, CRT allows us to decompose the problem into two simpler equations.

First, solve modulo 5:

$$x^2 \equiv 4 \pmod{5}.$$

The solutions are

$$x \equiv 2, 3 \pmod{5},$$

Next, solve modulo 7:

$$x^2 \equiv 4 \pmod{7}.$$

The solutions are

$$x \equiv 2, 5 \pmod{7},$$

We now combine the solutions pairwise using CRT.

For example, combine

$$x \equiv 2 \pmod{5}, \quad x \equiv 5 \pmod{7}.$$

Let

$$N_1 = 35/5 = 7, \quad N_2 = 35/7 = 5.$$

Compute modular inverses:

$$7^{-1} \equiv 3 \pmod{5}, \quad 5^{-1} \equiv 3 \pmod{7}.$$

By CRT,

$$x \equiv 2 \cdot 7 \cdot 3 + 5 \cdot 5 \cdot 3 = 117 \equiv 12 \pmod{35}$$

Therefore,

$$x \equiv 12 \pmod{35}.$$

Repeating this process for all combinations produces four solutions:

$$x \equiv 2, 12, 23, 33 \pmod{35}.$$

Thus, CRT transforms one equation modulo a composite number into several easier equations modulo its prime factors, and then reconstructs the final solutions.

3.4 Summary: Tractability of Problems in $\mathbb{Z}_n$

The security of systems built on composite moduli hinges entirely on the secrecy of the prime factors p and q . We can categorize computational problems in $\mathbb{Z}_n$ based on whether the factorization is known.

3.4.1 Easy Problems (Factorization Independent)

The following operations are efficient (polynomial time) even if the factorization of n is unknown:

- Generating random elements, addition, and multiplication.
- Computing modular exponentiation $g^r \pmod{n}$ using repeated squaring.
- Inverting elements using the Extended Euclidean Algorithm and solving linear systems.

3.4.2 Problems Reliant on Factorization

The following problems are believed to be intractable if the factorization of n is unknown, but become computationally trivial if p and q are known:

- **Testing Quadratic Residues:** Determining if an element is a QR in $\mathbb{Z}_n$.
- **Computing Square Roots:** Extracting the square root of a QR in $\mathbb{Z}_n$. This operation is provably as hard as factoring n .
- **Computing e 'th Roots:** Solving $x^e \equiv c \pmod{n}$ (where $\gcd(e, \phi(n)) = 1$), which is the core problem of breaking RSA without the private key.
- **Solving Polynomial Equations:** Finding roots for any equation of degree $d > 1$.

3.4.3 Globally Intractable Problems

The following problems remain hard in $\mathbb{Z}_n$:

- **The Factoring Problem:** Given a composite n , find its prime factors p and q . The best known algorithm, the General Number Field Sieve (GNFS), operates in sub-exponential time $\exp(\tilde{O}(\sqrt[3]{n}))$. Factoring a modern 1024-bit integer is approximately 1000 times harder than the current world record (the RSA-768 challenge).
- **Discrete Logarithm Problem (DLOG):** Given $x \in \mathbb{Z}_n^*$, finding an r such that $x \equiv g^r \pmod{n}$.
- **Diffie-Hellman Problem (DH):** Given g^{r_1} and g^{r_2} , computing $g^{r_1 r_2} \pmod{n}$.

References

- [1] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. Self-published, 2023. pages 2